

TRAINEE ONBOARDING GUIDE

Welcome to the

Microservices Course

Spring Boot & Spring Cloud
Professional Edition
Everything you need to do BEFORE Session 1

Duration	80 Hours · 29 Sessions · ~8 Weeks
Schedule	Wednesday offline (5h) + Monday/Wednesday online (2.5h each)
Platform	Enterprise E-Commerce — you build this from Session 1
Repository	microservices-pro-platform https://github.com/SayedBaladoh/microservices-pro-platform.git
Language	English materials · Arabic/English instruction

How to Use This Guide

Complete everything in this guide BEFORE Session 1. The first session assumes your environment is fully set up and working. We will not spend Session 1 time on installation.

Estimated Time to Complete This Guide

Step 1 — Read the course overview:	15 minutes
Step 2 — Install required tools:	45–60 minutes
Step 3 — Clone and verify the repository:	15 minutes
Step 4 — Run the infrastructure:	10 minutes
Step 5 — Verify everything works:	10 minutes

Total: approximately 1.5–2 hours

If you run into issues: contact your trainer BEFORE Session 1, not during.

1. What You Will Build

This is not a course of disconnected exercises. From Session 1 to Session 29, you will build ONE system that grows in capability every session.

Enterprise E-Commerce Platform

A production-grade microservices system you will design, build, secure, test, and deploy.

Services You Will Build

Service	Added in	What it does
Config Server + Eureka	Session 1	Infrastructure: centralized configuration + service discovery
API Gateway	Session 2	Single entry point: JWT auth, rate limiting, routing
Product Service	Session 1	Product catalogue: CRUD + Redis caching (Session 8)
Order Service	Session 4	Order management: Saga orchestration + resilience patterns
Payment Service	Session 4	Payment processing: Circuit Breaker, Retry, Fallback
Inventory Service	Session 6	Stock management: OpenFeign sync communication
Notification Service	Session 7	Event-driven: consumes Kafka events, sends notifications

Technology Stack

Category	Technologies	Introduced in
Core Framework	Spring Boot 3.x + Spring Cloud	Sessions 1-8
Service Discovery	Eureka (Spring Cloud Netflix)	Session 1
API Gateway	Spring Cloud Gateway + WebFlux	Sessions 2-3
Resilience	Resilience4j (CB, Retry, Bulkhead, TL)	Sessions 4-5
Communication	OpenFeign + Apache Kafka	Sessions 6-7
Caching	Redis + Spring Cache	Session 8
Containers	Docker + Docker Compose	Session 9
Orchestration	Kubernetes + Helm	Sessions 15-16
CI/CD	GitHub Actions + ArgoCD	Sessions 13-14
Observability	OpenTelemetry + Prometheus + Grafana + Jaeger	Session 17
Security	Keycloak + Istio + HashiCorp Vault	Sessions 20-21
Database	PostgreSQL + MongoDB	Sessions 1-7

2. Prerequisites — Do You Belong Here?

Be honest. This course is designed for experienced Java/Spring Boot developers. If you are missing the required skills, you will struggle from Session 1.

Required Skills (You Must Have These)

Skill	Level	What We Assume
Java	Strong	OOP, generics, lambdas, streams, checked/unchecked exceptions
Spring Boot	Strong	REST controllers, dependency injection, JPA, application.yml, Actuator
RESTful APIs	Strong	HTTP methods, status codes, request/response bodies, Postman testing
SQL / Relational DB	Solid	SELECT, JOIN, INSERT, UPDATE, basic schema design
Maven or Gradle	Solid	Add dependencies, run mvn test, understand pom.xml
Git	Solid	clone, branch, commit, push, pull request — daily use
Docker	Basic	Pull images, run containers, understand what Docker is for
Linux CLI	Basic	Navigate directories, run commands, read logs

❑ **If you are uncertain about Java or Spring Boot:** Complete a Spring Boot refresher course before Session 1. This course does not teach Spring Boot basics — it builds on them.

3. Install Required Tools

Install every tool in the order listed. Verify each one before moving to the next.

Tools Overview

Tool	Version	Verify Command	Windows Download	macOS / Linux
JDK 21	21.x LTS	<code>java -version</code>	adoptium.net	sdkman: <code>sdk install java 21</code>
Maven	3.9+	<code>mvn -version</code>	maven.apache.org	brew install maven
Docker Desktop	4.x+	<code>docker --version</code>	docker.com/desktop	docker.com/desktop
Git	2.x+	<code>git --version</code>	git-scm.com	brew install git
IntelliJ IDEA	2023.x+	Help -> About	jetbrains.com	jetbrains.com
Postman	Latest	Open app	postman.com	postman.com
kubectl	1.28+	<code>kubectl version</code>	kubernetes.io/docs	brew install kubectl

Step-by-Step Installation

Step 1 — Install JDK 21

1. Go to <https://adoptium.net> and download Eclipse Temurin JDK 21 (LTS)
2. Install using the default settings
3. Verify in a new terminal window:

```
$ java -version
openjdk version "21.0.x" ...

$ javac -version
javac 21.0.x
```

Common Issue: Wrong Java Version

If you see Java 11 or Java 17 instead of 21:

Windows: Set JAVA_HOME to the JDK 21 installation path in System Environment Variables

macOS/Linux: Add to `~/.zshrc` or `~/.bashrc`:

```
export JAVA_HOME=$(/usr/libexec/java_home -v 21)
```

```
export PATH=$JAVA_HOME/bin:$PATH
```

Then run: `source ~/.zshrc` and verify again with `java -version`

Step 2 — Install Maven

4. Download Maven 3.9+ from <https://maven.apache.org/download.cgi>
5. Extract and add to PATH (or use `brew install maven` on macOS)
6. Verify:

```
$ mvn -version
Apache Maven 3.9.x ...
Java version: 21.0.x ...
```

Step 3 — Install Docker Desktop

7. Download from <https://www.docker.com/products/docker-desktop>
8. Install and START Docker Desktop — the whale icon must be visible in the system tray
9. Verify:

```
$ docker --version
Docker version 24.x.x ...

$ docker compose version
Docker Compose version v2.x.x

# Quick smoke test
$ docker run hello-world
Hello from Docker!
```

❑ **Important:** Docker Desktop must be RUNNING (not just installed) whenever you work on the course. Check that the Docker icon is active in your system tray before every session.

Step 4 — Install Git

10. Download from <https://git-scm.com> (Windows) or use brew install git (macOS)
11. Configure your identity:

```
$ git config --global user.name "Your Name"
$ git config --global user.email "your.email@example.com"
$ git config --global core.editor "code --wait" # VS Code, or nano, vim

$ git --version
git version 2.x.x
```

Step 5 — Install IntelliJ IDEA

12. Download from <https://www.jetbrains.com/idea/> (Community Edition is sufficient)
13. Install the following plugins (File > Settings > Plugins):
 - Spring Boot (usually pre-installed)
 - Docker
 - Kubernetes
 - SonarLint (code quality)
14. Set JDK 21 as the default SDK: File > Project Structure > SDK > Add JDK > select JDK 21 path

Step 6 — Install Postman

15. Download from <https://www.postman.com/downloads/>
16. Create a free account (needed to save collections)

17. We will provide a Postman collection for testing all API endpoints in microservices-pro-course

4. Repository Setup

You will work in one repository throughout the entire course. Every session ends with a commit and push to this repository.

4.1 Clone the Platform Repository

```
# Clone the platform repository
$ git clone https://github.com/SayedBaladoh/microservices-pro-platform.git
$ cd microservices-pro-platform

# Verify the repository structure
$ ls -la
# You should see:
# services/      platform/      deployment/    docs/
# docker-compose.yml  README.md

# Check available branches
$ git branch -a
# You should see: main (your working branch)
#                  reference (instructor version - read only)
```

4.2 Understand the Repository Structure

Path	Purpose
services/product-service	Product REST API — built in Session 1
services/order-service	Order management — built in Session 4
services/inventory-service	Stock management — built in Session 6
services/payment-service	Payment processing — built in Session 4
services/notification-service	Event-driven notifications — built in Session 7
platform/config-server	Centralized config — built in Session 1
platform/eureka-server	Service discovery — built in Session 1
platform/api-gateway	Gateway + security — built in Sessions 2-3
deployment/docker/docker-compose.yml	Infrastructure: Kafka, Redis, PostgreSQL, Zipkin
deployment/kubernetes/	Kubernetes manifests — used in Sessions 15-16
reference (branch)	Instructor reference implementation — compare when stuck

The reference Branch — Your Lifeline

If you get stuck during a lab, the reference branch contains the instructor's complete implementation.

To compare your code with the reference:

```
$ git fetch origin
```

```
$ git diff main origin/reference -- services/product-service/
```

To view a file in the reference branch without switching:

```
$ git show origin/reference:services/product-service/src/main/.../ProductService.java
```

Do NOT merge or checkout the reference branch into main.

Use it as a reference only — your learning comes from writing the code yourself.

5. First Run — Verify Your Environment

Run through these steps in order. Every step must succeed before moving to the next. Do this at least 24 hours before Session 1 so your trainer has time to help if something fails.

5.1 Start the Infrastructure

```
# Navigate to the repository root
$ cd microservices-pro-platform

# Start all infrastructure services
$ docker compose up -d

# Wait 30 seconds, then check all containers are healthy
$ docker compose ps

# Expected output — ALL services should show "running" or "healthy":
NAME                STATUS              PORTS
postgres            running            0.0.0.0:5432->5432/tcp
redis               running            0.0.0.0:6379->6379/tcp
kafka               running            0.0.0.0:9092->9092/tcp
zookeeper           running            0.0.0.0:2181->2181/tcp
zipkin              running            0.0.0.0:9411->9411/tcp

# If any container shows "exited" or "unhealthy":
$ docker compose logs <service-name> # view error details
```

5.2 Verify Each Service

Service	Verify Command	Expected Response
PostgreSQL	<code>docker exec -it postgres psql -U postgres -c "SELECT 1"</code>	Returns: ?column? = 1
Redis	<code>docker exec -it redis redis-cli ping</code>	Returns: PONG
Kafka	<code>docker exec -it kafka kafka-topics.sh --list --bootstrap-server localhost:9092</code>	Returns: (empty list or topic names)
Zipkin UI	Open <code>http://localhost:9411</code> in browser	Shows Zipkin web interface

5.3 Run a Quick Java Test

Verify that IntelliJ can build a Spring Boot project before Session 1:

18. Open IntelliJ IDEA
19. Go to File > New > Project > Spring Initializr
20. Select: Java 21 · Maven · Spring Boot 3.x

21. Add dependency: Spring Web
22. Click Create and wait for Maven to download dependencies
23. Run the main application class — it should start on port 8080
24. Open <http://localhost:8080/actuator/health> in a browser

Expected Output

```
{"status":"UP"}
```

If you see this — your Java + Maven + IntelliJ setup is working correctly.

If you see any error — contact your trainer before Session 1.

6. How the Course Works

6.1 Session Structure

Online Session (2.5 hours)	Offline Anchor Day (5 hours)
00:00 Cold Call on pre-reading (10 min) 00:10 Concept + live coding (60 min) 01:10 Lab work — you implement (35 min) 01:45 Debrief + Q&A (10 min) 01:55 Daily Quiz — 8 questions (10 min) 02:05 Checkpoint Commit + preview (5 min)	00:00 Deep theory + whiteboard (60 min) 01:00 BREAK (15 min) 01:15 Extended live coding (45 min) 02:00 LUNCH (45 min) 02:45 Full lab — supervised (90 min) 04:15 Architecture discussion (30 min) 04:45 Quiz + checkpoint (15 min)

6.2 The Checkpoint Commit — Non-Negotiable

Every Session Ends With a Commit and Push

At the end of every session, you must:

1. Complete the lab (or reach the defined checkpoint)
2. Ensure all unit tests pass: `mvn test`
3. Commit with the correct message format:
`session-NN: short-description-of-what-was-done`

Example commit messages:

`session-01: add-product-service-eureka-config`
`session-04: add-circuit-breaker-and-retry-on-order-payment`
`session-07: add-choreography-saga-order-inventory-payment`

4. Push to GitHub: `git push origin main`

Labs that are not committed are not graded.

If you cannot finish in session, complete it as homework before the next session.

6.3 Unit Tests Are Mandatory

Every lab requires unit tests. This is not optional. Your lab grade includes:

Component	Weight	What We Check
Feature works correctly	70%	Endpoints respond, business logic correct
Unit tests present and passing	20%	<code>mvn test</code> = green, minimum test count met

Code quality	10%	Clean code, no obvious duplication
--------------	-----	------------------------------------

6.4 Pre-Session Reading

15 Minutes That Change the Session Quality

Every session has required reading: 15 minutes, focused, specific.

The reading is listed in the Session Pack under "Pre-Session Reading".

At the START of every session, your trainer will ask 3 Cold Call questions on the reading.
They will call on specific people — not wait for volunteers.

You are expected to have read the material.

If you have not read it: answer honestly and explain what you understood.

Do NOT say "I did not have time" — that is the wrong culture for this course.

Trainees who read consistently get significantly more from every session.

6.5 Assessment Summary

Component	Details
Daily Quizzes (10%)	8 questions at the end of each session · auto-graded
Lab Grades (40%)	Feature + tests + code quality · per lab rubric
Mid-Course Exam (15%)	Session 9 · 2 hours · individual · practical coding exam
Capstone Project (35%)	Team 70% + Individual Contribution 30%
Distinction	90–100%
Pass with Merit	75–89%
Pass	60–74%
Retake Required	Below 60%

7. Before Session 1 — Final Checklist

Tick every item. If anything is not ticked, resolve it before Session 1.

Environment

- ☐ JDK 21 installed and verified: `java -version` shows 21.x
- ☐ Maven 3.9+ installed: `mvn -version` shows Maven + Java 21
- ☐ Docker Desktop installed, running, and `docker run hello-world` succeeds
- ☐ Git installed and configured with name + email
- ☐ IntelliJ IDEA installed with Spring Boot + Docker plugins
- ☐ Postman installed and account created

Repository

- ☐ `microservices-pro-platform` repository cloned successfully
- ☐ `git branch -a` shows both main and origin/reference
- ☐ `docker compose up -d` runs without errors
- ☐ All Docker containers show "running": postgres, redis, kafka, zipkin
- ☐ Redis responds: `docker exec -it redis redis-cli ping` returns PONG

IDE & Build

- ☐ IntelliJ opens the `microservices-pro-platform` folder without errors
- ☐ JDK 21 set as default SDK in IntelliJ (File > Project Structure)
- ☐ Quick Spring Boot test project created, started, and `/actuator/health` returns UP

Course Preparation

- ☐ Pre-Session 1 reading completed (see below)
- ☐ Trainer contact information saved
- ☐ Course schedule added to calendar — all 29 session dates
- ☐ Committed to completing labs between sessions (homework is expected)

Pre-Session 1 Required Reading

Read These Before Your First Session (30 minutes total)

1. Microservices Architecture Overview — Martin Fowler:
<https://martinfowler.com/articles/microservices.html>
Read: Introduction + "Characteristics of a Microservice Architecture" sections
2. 12-Factor App — <https://12factor.net>

Read: All 12 factors (5 minutes each — they are short)

3. Spring Cloud Overview:

<https://spring.io/projects/spring-cloud>

Read: Overview + project list (just names and one-line descriptions)

Focus questions — your trainer will ask these at the start of Session 1:

Q1: What is the main problem that Microservices solve compared to a Monolith?

Q2: Which 12-Factor principle does Spring Cloud Config Server directly implement?

Q3: Name three Spring Cloud projects and what problem each one solves.

8. Quick Reference — Commands You Will Use Every Session

Docker Compose

```
# Start all infrastructure
docker compose up -d

# Check container status
docker compose ps

# View logs for a specific service
docker compose logs kafka
docker compose logs -f kafka # follow mode

# Stop all infrastructure
docker compose down

# Stop and remove volumes (clean state)
docker compose down -v
```

Git — Session Workflow

```
# Check what changed
git status
git diff

# Stage all changes
git add .

# Commit with the required format
git commit -m "session-04: add-circuit-breaker-and-retry-on-order-payment"

# Push to GitHub
git push origin main

# View the reference branch (DO NOT modify it)
git fetch origin
git show origin/reference:services/order-service/src/main/.../OrderService.java
```

Maven

```
# Compile and run all tests
mvn test

# Build without running tests
mvn package -DskipTests

# Run a Spring Boot application
mvn spring-boot:run
```

```
# Clean build artifacts
mvn clean

# Clean and rebuild
mvn clean package
```

Key URLs — Available From Session 1

URL	Service
http://localhost:8761	Eureka Dashboard — registered services
http://localhost:8888/actuator/health	Config Server health check
http://localhost:9411	Zipkin distributed tracing UI
http://localhost:6379	Redis (use redis-cli, not browser)
http://localhost:5432	PostgreSQL (use IntelliJ database tool)
http://localhost:9092	Kafka broker (use kafka-console-consumer)

You are ready. See you in Session 1.

Wednesday · 10:00 AM · In Person · 5 Hours

microservices-pro-course · microservices-pro-platform · ITSharks · Dr Sayed Baladoh